INDUSTRY 4.0 · LIVE DEMONSTRATION

Smart Factory Assistant

Wokwi demonstration of AI-based predictive maintenance and intelligent job scheduling

Daniel Chidi Chimezie · **Roland Bukoola**

Programming II · Prof. Martin Sonntag · Hochschule Hamm-Lippstadt

OBJECTIVE

Demonstrating a virtual smart factory

Show how industrial sensors monitor machine health, predict failures, and intelligently schedule manufacturing jobs in a live Industry 4.0 simulation.

01 **Monitor** Track temperature and vibration in real time

02 **Predict** Calculate a health score and forecast failures

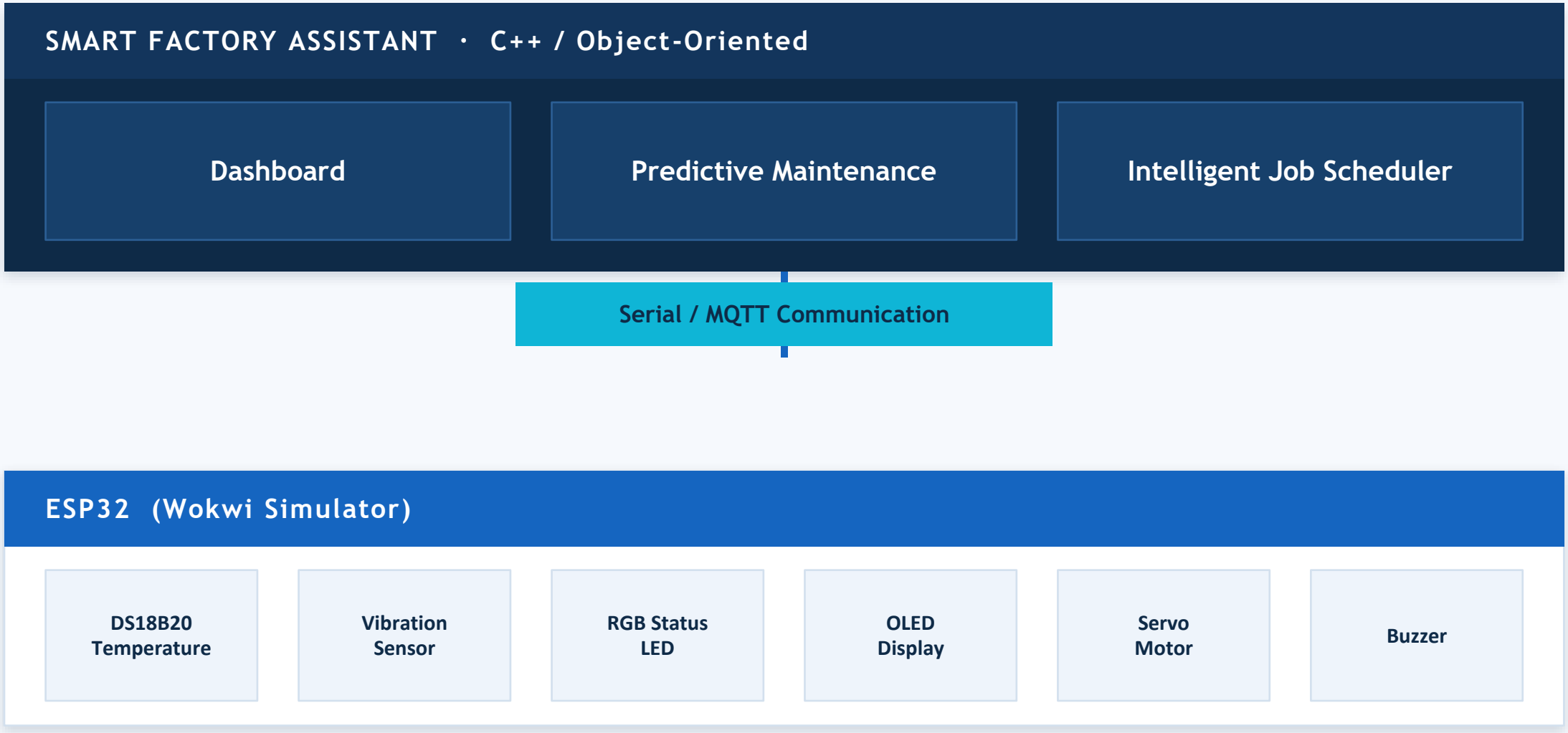
03 **Alert** Raise maintenance warnings before breakdowns

04 **Schedule** Assign jobs to the healthiest available machine

Automated Industry 4.0 production line

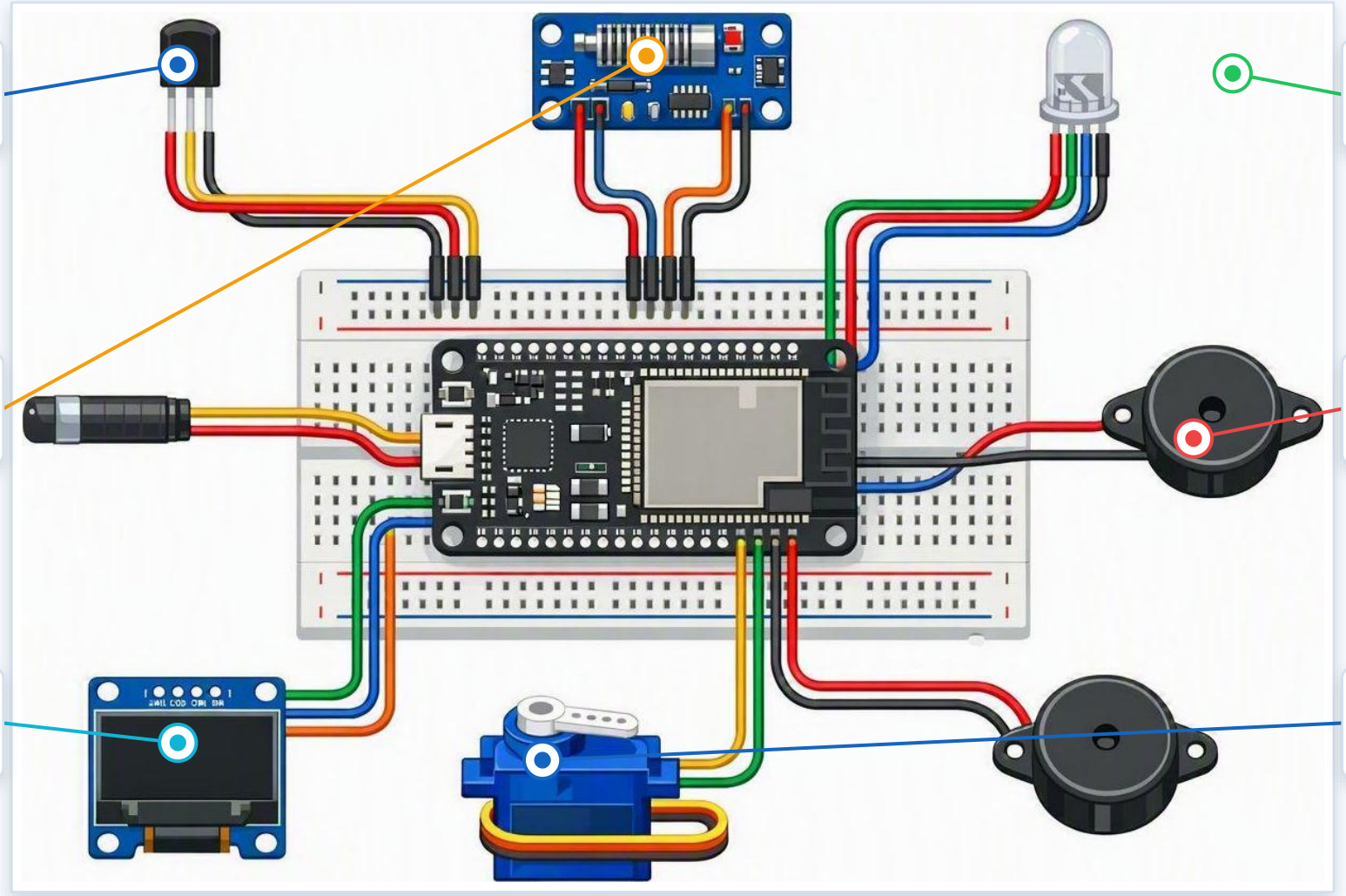
SYSTEM ARCHITECTURE

From sensors to smart decisions



WOKWI WIRING

Components wired to the ESP32 — GPIO callouts



DS18B20 Temp.
GPIO 4 · 1-Wire

Vibration Sensor
GPIO 34 · analog

OLED Display
GPIO 21/22 · I²C

RGB Status LED
GPIO 25/26/27

Buzzer
GPIO 14 · digital

Servo Motor
GPIO 13 · PWM

WOKWI COMPONENTS

Hardware that powers the simulation

Component	Purpose
ESP32	Main controller for data acquisition
DS18B20	Measures machine temperature
Vibration Sensor	Detects abnormal machine vibration
RGB LED	Indicates machine health — green, yellow, red
OLED Display	Shows machine status and health score
Buzzer	Sounds an alarm during critical failures
Servo Motor	Simulates machine operation

7

simulated components

Status colours

- Green — healthy
- Yellow — warning
- Red — critical

PREDICTIVE MAINTENANCE

How the health score drives decisions

HEALTH SCORE

Health Score = 100 − (0.4 × Temperature Risk + 0.6 × Vibration Risk)

A lower score signals a higher likelihood of failure — alerts fire when thresholds are exceeded.

Healthy

Score 70-100

Green LED · servo running · jobs assigned freely

Warning

Score 40-69

Yellow LED · maintenance alert raised · monitor closely

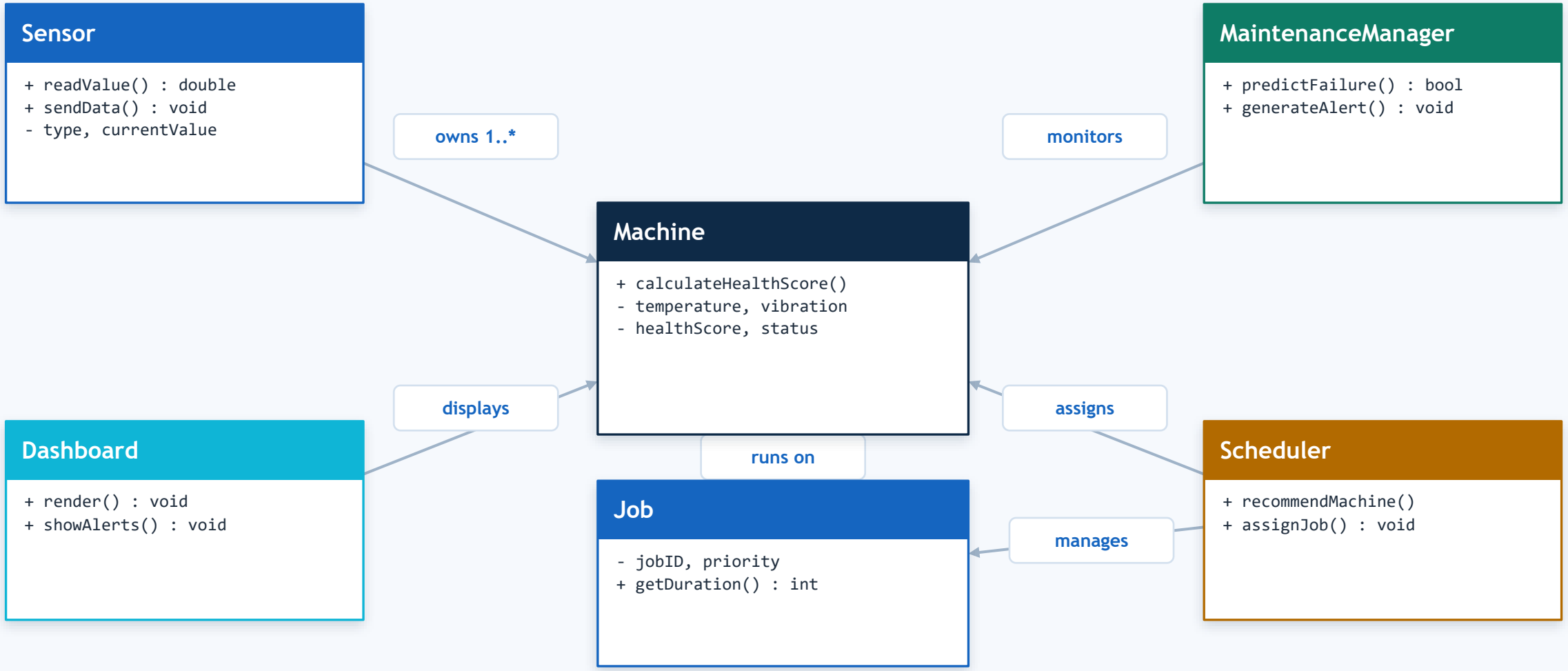
Critical

Below 40

Red LED · buzzer alarm · machine excluded from scheduling

CLASS DESIGN

How the six C++ classes relate



C++ IMPLEMENTATION · 1 OF 3

Machine & Sensor classes

Each machine encapsulates its sensor readings and computes its own health score from the weighted risk formula.

```cpp
class Sensor {
public:
    int         sensorID;
    string      type;        // "temperature" | "vibration"
    double      currentValue;
    double readValue();      // poll the hardware pin
    void   sendData();        // push over Serial / MQTT
};

class Machine {
public:
    int    machineID;   string status;  // Healthy|Warning|Critical
    double temperature, vibration, healthScore;

    double calculateHealthScore() {
        double tRisk = clamp((temperature - 40.0) / 40.0);
        double vRisk = clamp(vibration / 5.0);
        healthScore = 100.0 - (0.4*tRisk + 0.6*vRisk) * 100.0;
        return healthScore;
    }
};
```

C++ IMPLEMENTATION · 2 OF 3

Predictive maintenance & scheduling

The maintenance manager flags failing machines, and the scheduler routes each job to the healthiest one available.

```cpp
class MaintenanceManager {
public:
    bool predictFailure(const Machine& m) {
        return m.healthScore < 40.0;   // critical threshold
    }
    void generateAlert(const Machine& m) {
        if (m.healthScore < 70.0)
            cout << "[ALERT] machine " << m.machineID;
    }
};

class Scheduler {
public:
    // assign the job to the healthiest eligible machine
    Machine* recommendMachine(vector<Machine>& pool) {
        Machine* best = nullptr;
        for (auto& m : pool)
            if (m.healthScore >= 70.0 &&
                (!best || m.healthScore > best->healthScore))
                best = &m;
        return best;
    }
};
```

C++ IMPLEMENTATION · 3 OF 3

ESP32 firmware in Wokwi

On the device, the ESP32 reads the sensors, drives the status outputs, and streams JSON telemetry to the assistant.

```cpp
// ESP32 firmware (Wokwi) - read sensors, drive outputs
#include <OneWire.h>
#include <DallasTemperature.h>

OneWire oneWire(4);
DallasTemperature ds18b20(&oneWire);

void loop() {
    ds18b20.requestTemperatures();
    float tempC = ds18b20.getTempCByIndex(0);
    int   vib   = analogRead(34);        // vibration

    float health = computeHealth(tempC, vib);
    setStatusLed(health);            // green / yellow / red
    if (health < 40) tone(BUZZER, 2000); // alarm

    Serial.printf("{\"temp\":%.1f,\"health\":%.0f}\n",
                  tempC, health);  // -> Assistant
}
```

OOP PRINCIPLES

Object-oriented design in action

1 Encapsulation

Each Machine hides its raw sensor data and exposes only calculateHealthScore() — internal state stays private.

2 Abstraction

The Sensor interface defines readValue() and sendData(); callers never touch the underlying hardware details.

3 Inheritance

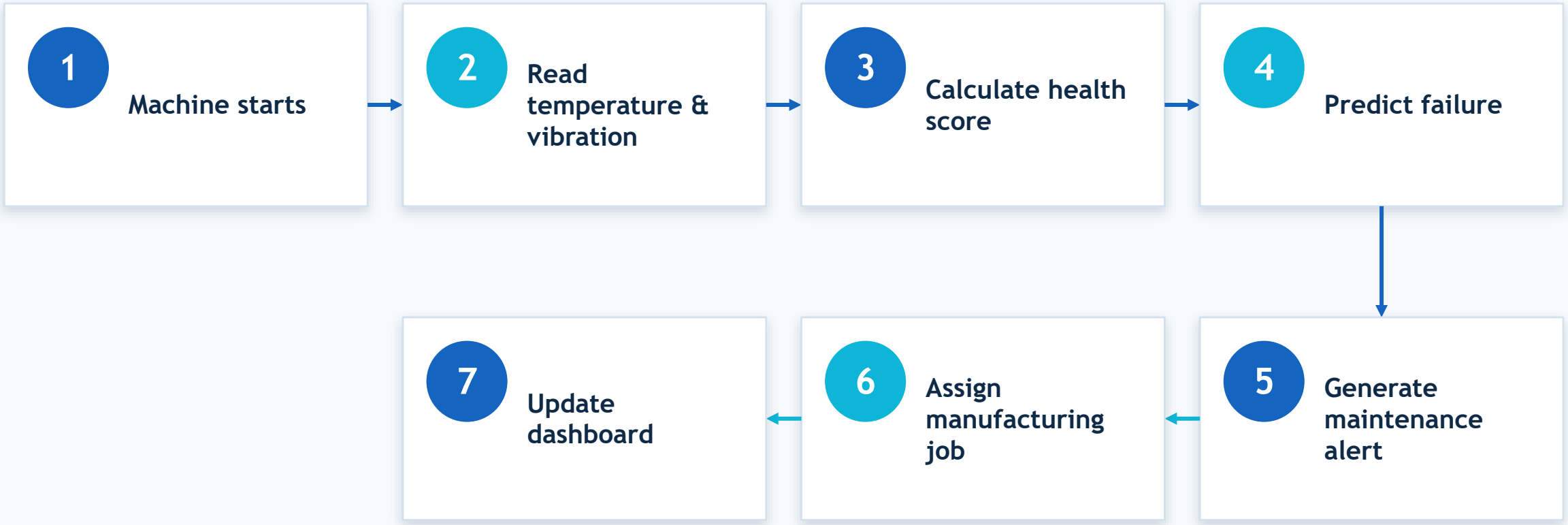
TemperatureSensor and VibrationSensor extend a shared Sensor base class, reusing its common contract.

4 Polymorphism

A single readValue() call resolves to the correct sensor implementation at runtime through a base pointer.

DEMONSTRATION WORKFLOW

From machine start to dashboard update



LIVE DEMO SCENARIO

Watch a machine overheat — then recover

STEP 1
Normal

HEALTH

92

Temp **38 °C**

Green LED

Servo running · jobs assigned freely.

STEP 2
Heating up

HEALTH

64

Temp **58 °C**

Yellow LED

Maintenance alert raised · monitor closely.

STEP 3
Critical

HEALTH

28

Temp **78 °C**

Red LED

Buzzer alarm · machine paused immediately.

STEP 4
Recovery

HEALTH

↑

Temp —

Reschedule

Scheduler routes the job to a healthier machine.

RESULTS & STACK

Expected output and technologies

Expected output

- ✓ Live sensor monitoring
- ✓ Machine health calculation
- ✓ Predictive maintenance alerts
- ✓ Automatic job scheduling
- ✓ Industry 4.0 smart factory simulation

Technologies

C++ | OOP | ESP32 | Wokwi Simulator | IoT Sensors | Predictive Maintenance | Intelligent Scheduling

Object-oriented C++ classes — Machine, Sensor, MaintenanceManager, Job, Scheduler and Dashboard — drive the simulation logic.

WHY IT MATTERS

Less downtime, smarter production

Predictive maintenance catches failures early, while intelligent scheduling keeps jobs flowing to the healthiest machines — a practical Industry 4.0 application built on object-oriented C++.

Daniel Chidi Chimezie · Roland Bukoola
Programming II · Hochschule Hamm-Lippstadt